



SCHOOL OF ADVANCED TECHNOLOGY
SAT301 FINAL YEAR PROJECT

*OptNet-Based Constrained Imitation Learning
for Cart-Pole Control*

Final Thesis

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Engineering

Student Name	:	Enze Jin
Student ID	:	2253685
Supervisor	:	Kai Pfeiffer
Assessor	:	

Abstract

This thesis studies a constrained imitation-learning controller for the cart pole control task. A normal supervised model can be trained to map the current cart pole state to the expert control input, but this kind of model mainly works as a direct predictor. It may fit the training labels well, but the physical limits of the system, such as the force range and cart-position bound, are not explicitly included in the model output.

To make these constraints part of the learning structure, this work develops an OptNet-based controller with a differentiable quadratic programming layer. Expert trajectories are first generated and used as supervised demonstrations. After filtering and normalization, a baseline MLP is trained for comparison. The proposed OptNetCartPoleMPC model then uses a short-horizon QP layer to predict future state-control variables under equality and inequality constraints. The equality constraints describe a simplified predictive dynamics model, while the inequality constraints represent the control-force and cart-position limits. Only the first control input from the QP solution is used as the final output, following the receding-horizon idea.

The main evaluation is not limited to one-step prediction accuracy. This is important because a controller that looks accurate on a dataset may still fail when it is rolled out over time. Therefore, the models are compared using prediction metrics, rollout behaviour, constraint violation count, success rate, horizon comparison, and QP matrix visualization. The results show that the OptNet-QP model keeps the cart position within the tested boundary, while the MLP baseline violates the same boundary during rollout. In the main rollout comparison, the OptNet-QP model produces 0 constraint violations, whereas the MLP baseline produces 9386 violations out of 30000 evaluated rollout steps.

The horizon comparison also shows that validation loss alone is not enough to select the best controller. Although $N = 10$ gives the lowest validation loss, $N = 12$ achieves the highest success rate of 40.00% while keeping the cart-position violation rate at 0.00%. This suggests that, for this task, the prediction horizon should be chosen according to closed-loop rollout behaviour rather than supervised loss alone.

Keywords: OptNet; cart-pole control; constrained learning; quadratic programming; imitation learning; differentiable optimization

Acknowledgements

I would like to thank my supervisor, Kai, for guiding, supporting and being patient with me throughout this final year project. His feedback helped me to better understand the direction of this project and to gradually clarify the link between supervised learning, constrained control and the OptNet-based QP model. The discussions with him helped me to improve both the technical design and the evaluation of this work.

I would also like to thank my parents for their continuous encouragement and support during my undergraduate study. Their understanding gave me the confidence to keep working through the difficulties I met during this project.

Finally, I would like to thank my classmates and friends who helped and supported me during the project. The discussions, suggestions, and mutual encouragement among us made the process less stressful and more meaningful.

Contents

Abstract.....	ii
Acknowledgements	iii
Contents	iv
List of Tables.....	vi
List of Figures	vii
Chapter 1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	2
1.3 Aim and Objectives.....	3
Chapter 2 Literature Review	4
2.1 Cart-Pole Control and Constrained Control	4
2.2 Supervised Learning for Control	6
2.3 Constraints in Supervised Learning	7
2.4 Differentiable Optimization Layers and OptNet	9
2.5 Differentiable MPC and Receding-Horizon Learning	10
2.6 Data Preprocessing for Supervised Control	11
Chapter 3 Methodology	12
3.1 Overall Project Pipeline.....	13
3.2 Data Generation	14
3.3 Data Preprocessing and Normalization	17
3.4 OptNet Model Design	18
3.5 Baseline Neural Network	22
3.6 Training and Evaluation Setup	23
Chapter 4 Results and Evaluation.....	26
4.1 Training Results	26
4.2 Prediction Accuracy	30
4.3 Rollout Performance	31
4.4 Success Rate Evaluation and Prediction-Horizon Comparison.....	33
4.5 QP Constraint Verification	36
Chapter 5 Conclusion and Future Work	40
5.1 Conclusion	40
5.2 Limitations.....	42
5.3 Future Work.....	43
References	45

List of Tables

Table 1. Problem setting of the cart-pole control task	3
Table 2. Training hyperparameters	24
Table 3. Prediction accuracy of the OptNet-QP model in physical units	31
Table 4 . Cart position constraint violations under the ± 1.0 m constraint	32
Table 5. Performance comparison of OptNet-QP models with different prediction horizons.....	34

List of Figures

Figure 1. Generated cart-pole trajectories used for supervised learning.	16
Figure 2. Structure of the proposed OptNetCartPoleMPC model	19
Figure 3. Structure of the baseline MLP	22
Figure 4. Training, validation, and test loss curves of the OptNet-QP model	27
Figure 5. Loss comparison between OptNet-QP and MLP baseline.	28
Figure 6. Validation loss comparison among constrained OptNet-QP, relaxed-bound OptNet-QP, and MLP baseline.	29
Figure 7. Rollout comparison between OptNet and MLP under the ± 1.0 m cart position constraint.....	32
Figure 8. Sparse structure of the inequality constraint matrix G	38
Figure 9. Inequality constraint vector h for the 10-step OptNet-QP model	39

Chapter 1

Introduction

1.1 Background and Motivation

Control problems often require a controller to make decisions while satisfying physical constraints. In many real systems, the control input cannot be unlimited, and the system states also need to remain within safe or meaningful ranges. The cart pole system is a typical example of this type of problem. Although the system appears simple, it still contains nonlinear dynamics, coupling between the cart and the pole, and clear physical limits such as the maximum control force and the allowable range of cart movement.

The two constraints considered in this work are the cart position bound and the control force limit. In the constrained data setting, the cart position is kept within $[-1, 1]$ m, and the applied control force is bounded within $[-10, 10]$ N. These limits are important because the controller should not only imitate the expert force value, but also produce behaviour that remains physically admissible.

A common way to train a controller from data is supervised imitation learning. In this setting, expert or reference trajectories are generated first, and a model then learns the mapping from system state to control action [1]. This is the approach used in this project. Once the model has been trained, it can output a control force directly, without solving the original trajectory optimization problem at every time step.

The weakness of a standard neural network controller is that these physical limits are not part of the model structure. A multilayer perceptron may learn the relationship between the current state and the expert control action, but it does not explicitly encode the cart position bound or the force bound. For example, it does not naturally guarantee that the predicted force will stay within $[-10, 10]$ N, and it also does not directly encode the cart position bound. In other words, the model may achieve a low prediction error, but this does not always mean that it is suitable as a constrained controller. Previous studies on

constraint-based learning also point out that known rules, bounds, and domain knowledge can be useful when they are directly included in the learning structure rather than only learned from data [2], [3], [4].

Model Predictive Control provides a useful reference idea for this type of problem. MPC predicts future states and control inputs over a finite horizon, solves an optimization problem with dynamics and constraints, and usually applies only the first control action before repeating the process at the next time step [5], [6]. This idea is useful for the cart-pole task because the current control force affects not only the current state, but also the future movement of the cart and pole.

This project is motivated by the idea of combining supervised learning with constrained optimization. Instead of using a pure black-box neural network to directly predict the control input, this project uses an OptNet-based differentiable quadratic programming layer. OptNet shows that a quadratic program can be embedded as a layer inside a neural network and trained through backpropagation [7]. Related work on differentiable convex optimization layers further shows that optimization problems can be integrated into end-to-end learning systems [8]. In this project, the QP layer is used to include a linear short-horizon predictive structure, control force bounds, and cart position bounds inside the learned controller. The aim is not to build a full nonlinear MPC controller, but to develop a small MPC-inspired QP layer that can be trained end-to-end as part of a neural network policy.

1.2 Problem Statement

The main problem considered in this project is how to learn a cart-pole control policy from generated expert trajectories while also taking physical constraints into account. The input of the controller is the current cart-pole state,

$$\mathbf{x}_t = [x_t, \dot{x}_t, \theta_t, \dot{\theta}_t]^T$$

where x_t is the cart position, $\dot{x}_t$ is the cart velocity, θ_t is the pole angle, and $\dot{\theta}_t$ is the angular velocity. The output of the controller is the control input u_t , which represents the horizontal force applied to the cart.

Table 1. Problem setting of the cart-pole control task

Item	Description
Input state	$\mathbf{x}_t = [x_t, \dot{x}_t, \theta_t, \dot{\theta}_t]^T$
Output	Control force (u_t)
Main constraints	Cart position bound and control force bound
Baseline model	MLP state-to-control regression
Proposed model	OptNet-QP controller with explicit constraints
Main evaluation	Prediction accuracy, rollout behaviour, violation count, success rate

As shown in Table 1, the task is treated as a state to control learning problem. The model receives the current cart-pole state and predicts the horizontal control force. The baseline MLP learns this mapping directly, so it is useful as a simple reference model. However, it does not contain the cart position bound or the force limit as explicit parts of the model structure.

This creates the main difficulty studied in this thesis. A controller may achieve a low regression loss on the dataset but still behave poorly when it is rolled out over time. For example, the predicted force may be too large, or the repeated predictions may gradually push the cart outside the allowed position range. Therefore, the model should not be judged only by prediction error. Rollout behaviour and constraint satisfaction also need to be evaluated.

The main research question is whether adding a differentiable QP layer can make the learned controller more constraint aware and easier to interpret than a standard MLP baseline. In this comparison, the MLP represents a purely data driven state to control model, while the OptNet-QP controller represents a supervised model with embedded physical constraints.

1.3 Aim and Objectives

The aim of this project is to develop and evaluate an OptNet based constrained imitation learning controller for the cart-pole task. The controller is expected to learn from expert trajectories, while also keeping important physical constraints, such as cart position bounds and control force limits, inside the model structure.

To achieve this aim, the project first generates a cart-pole dataset containing system states and expert control inputs. The generated trajectories are then filtered and normalized before training, so that abnormal samples and scale differences have less influence on the learning process. A baseline MLP controller is trained as a reference model. In parallel, the proposed OptNetCartPoleMPC model is built by adding a differentiable QP layer to the neural network output. This QP layer includes the control force bound, cart position bound, and short horizon predictive constraints.

The two models are evaluated through prediction accuracy, rollout behaviour, constraint violation count, success rate, horizon comparison, and QP matrix visualization. The purpose of this comparison is to check whether the OptNet-QP structure can provide better constraint awareness and interpretability than a standard MLP controller.

Chapter 2

Literature Review

2.1 Cart-Pole Control and Constrained Control

The cart-pole system is a useful starting point for this project because it is simple to simulate but still difficult to control well. The motion of the cart and the pole is coupled: when a force is applied to the cart, it changes the cart position and velocity, and at the same time affects the pole angle and angular velocity. This coupling makes the system unstable and sensitive to the timing of the control input. Because of this, the task is suitable for testing whether a learned controller can handle dynamic behaviour rather than only fit a static input output relation.

In this thesis, the cart-pole task is treated as a constrained control problem rather than only a state-to-control regression problem. The controller needs to generate a suitable force from the current state, but the force and the cart position cannot be unlimited. In the experimental setting, the control force is bounded within $[-10, 10]$ N, and the cart is also required to remain inside a specified position range. A low prediction error alone is therefore not enough. If a learned policy produces an excessive force or gradually drives the cart outside the allowed region, it is still not a reliable constrained controller.

This is where constrained control differs from ordinary function approximation. A neural network can approximate the mapping from state to action, but it does not automatically know which actions are physically admissible. By contrast, optimization-based control methods can place the constraints directly inside the decision-making problem. For this reason, they provide a useful reference point for the OptNet-QP design used later in this work.

Model Predictive Control is one of the main frameworks for handling this type of constrained decision-making problem. In MPC, the controller predicts future states and control inputs over a finite horizon, solves an optimization problem with dynamics and constraints, and applies only the first control action before repeating the process at the next time step [5], [6]. This receding-horizon structure is relevant to the cart-pole task because the current force affects the future motion of both the cart and the pole, not only the immediate state.

Optimal control also provides the background for the data generation stage of this project. In a typical optimal control problem, a sequence of states and control inputs is optimized by minimizing a cost function while satisfying the system dynamics and possible constraints [6]. For the cart-pole system, this means that the trajectory should move the pole towards the desired upright configuration, while still keeping the cart position and the applied force within acceptable limits.

In this project, trajectory optimization is used to generate the expert behaviour before training the neural controllers. The aim is not to learn a feedback policy at this stage, but to obtain complete state and control sequences that can later serve as supervised demonstrations. Differential Dynamic Programming (DDP) provides a classical way to

solve nonlinear optimal control problems by repeatedly improving a control sequence using local approximations of the dynamics and cost function [9]. Since the present task also includes bounds on state and control variables, the constrained extension used for data generation is discussed next.

For constrained control problems, standard DDP needs to be extended so that state and control constraints can be handled more directly. ProxDDP is one example of this direction. It combines DDP-type trajectory optimization with proximal constrained optimization ideas, allowing equality and inequality constraints to be included in the trajectory optimization process [10]. In this project, ProxDDP is not used as the final online controller. Instead, it is used offline through `aligator` to generate expert cart-pole trajectories. These optimized trajectories are then used as supervised learning data for the baseline MLP and the proposed OptNet-QP controller.

This project therefore sits between optimal control and supervised learning. The expert trajectories are generated from an optimization-based control method, while the final controller is learned from these trajectories. The baseline MLP learns the state-to-control mapping directly, whereas the OptNet-QP model adds a constrained optimization layer to the neural network output. This design does not aim to replace full nonlinear optimal control. Instead, it uses optimal-control-generated demonstrations as data and uses a QP layer to impose explicit state and control constraints in the learned controller.

2.2 Supervised Learning for Control

Supervised learning can be used for control by learning a mapping from the current system state to the corresponding control input. In this project, the input state is defined as

$$\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]^T$$

where x is the cart position, $\dot{x}$ is the cart velocity, θ is the pole angle, and $\dot{\theta}$ is the pole angular velocity. The model is trained to predict the control input u from this state. This is a direct and practical way to train a controller from generated expert trajectories.

This type of method is closely related to imitation learning, where a model learns a policy by imitating expert demonstrations or reference trajectories. Guided policy search

also shows how supervised learning can relate to policy learning by using trajectory-based information to guide the training process [1]. In this project, the expert trajectories are generated first, and the neural network controller is then trained to imitate the corresponding control inputs.

A standard multilayer perceptron is a natural baseline for this kind of task. It is simple to implement, fast to evaluate, and can learn the general relationship between the state and the control input. Once trained, an MLP can produce control actions quickly without solving a full trajectory optimization problem online. This makes it a useful comparison model in this project.

However, the main limitation of an ordinary MLP is that it works mostly as a black-box function approximator. It can reduce the prediction loss on the training and testing data, but it does not explicitly include the control force bound, the cart position bound, or the short-term dynamics of the system. Therefore, a low regression error does not necessarily mean that the controller will behave safely during rollout. For example, small one-step prediction errors may accumulate over time and lead to the cart outside the allowed position range.

This limitation shows why a constrained learning structure is useful. Instead of relying only on the dataset to teach the model what is physically reasonable, part of the control knowledge can be placed directly into the model structure. Previous work on constraint-based domain knowledge points out that constraints can improve deep learning models when the task has known rules, output bounds, or physical relationships [2]. In addition, surveys on constrained optimization learning also show that optimization layers can be used to connect machine learning models with structured decision-making problems [3].

2.3 Constraints in Supervised Learning

A standard supervised learning model is usually trained by minimizing the error between the predicted output and the target label. This is effective when the main goal is prediction accuracy. However, in many control and decision-making problems, prediction accuracy alone is not sufficient. The model output should also satisfy known constraints,

such as physical limits, safety bounds, or structural rules. In the cart-pole task, this means that the predicted control force should stay within the actuator limit, and the cart position should remain within the allowed physical range.

For this project, supervised control cannot be judged only by how close the predicted action is to the expert label. The learned action also needs to respect the physical limits of the cart-pole system. One way to deal with this issue is to bring known restrictions into the learning process, instead of expecting the network to discover them only from data. These restrictions may appear as extra terms in the loss function, or as part of the model structure itself. This idea is also discussed by Borghesi et al., who show that constraint based domain knowledge can guide deep learning models when a task contains known rules or bounds [2]. Kotary et al. also describe constrained optimization learning as a wider framework for connecting machine learning with structured decision-making problems [3].

One possible approach is to add a constraint-related penalty to the loss function. For example, Xu et al. proposed a semantic loss function to connect neural network outputs with logical constraints [11]. The model is still trained from data labels, but the loss also measures whether the output is close to satisfying the given rules. Although semantic loss is mainly designed for symbolic or logical constraints, it is still relevant here because it shows a similar idea: supervised learning can be guided by information that is not contained only in the input-output pairs.

The method used in this thesis is closer to the second approach, where constraints are imposed through the output structure. Instead of only adding a penalty term and hoping that the network learns constraint-satisfying behaviour, the OptNet-QP model obtains its control output by solving a constrained quadratic program. In simple terms, the force limit and cart-position bound are not just examples seen in the training data; they are written into the QP formulation itself.

This design also has to be treated carefully. Hard constraints can improve the structure of a neural network model, but they are only useful when the constraints are correctly formulated and compatible with the learning problem. Márquez-Neila et al. point out that imposing hard constraints on deep networks has both benefits and limitations [4]. This is relevant to the present work because the OptNet layer can only enforce the constraints

included in its QP. It does not automatically solve every control difficulty. For example, the internal predictive model is simplified, and the QP solution can still be affected by numerical conditioning.

For this reason, the constrained structure in this thesis should be understood as a practical middle ground. It does not replace full nonlinear optimal control, but it gives the supervised controller a clearer physical structure than a standard MLP. The model is still trained using expert state-control data, but the final action is produced through a constrained optimization layer. This also makes the controller easier to inspect, since the constraint matrices can be checked directly rather than hidden inside the weights of a black-box network.

2.4 Differentiable Optimization Layers and OptNet

Differentiable optimization layers provide a way to embed optimization problems inside neural networks. Instead of using a conventional neural network layer that applies a simple affine transformation and activation function, an optimization layer outputs the solution of an optimization problem. This allows the model to include constraints and structured decision-making directly in the forward pass.

OptNet is one of the key works in this area. Amos and Kolter proposed using quadratic programs as differentiable layers inside neural networks [7]. In an OptNet layer, the forward pass solves a QP, and the backward pass differentiates through the solution by using the optimality conditions of the QP. This makes it possible to train the optimization layer together with the rest of the neural network.

The general QP form used in OptNet can be written as

$$\min_z \frac{1}{2} z^T Q z + q^T z$$

subject to

$$Az = b, Gz \leq h$$

where z is the optimization variable, Q and q define the objective function, A and b define equality constraints, and G and h define inequality constraints. This structure is

suitable for this project because the short-horizon linear predictive relationship can be written as equality constraints, while the control force and cart position bounds can be written as inequality constraints.

The idea of differentiable optimization layers was later extended beyond the original OptNet formulation. Agrawal et al. showed that more general convex optimization problems can also be placed inside end to end learning systems [8]. This is relevant to the present project because the controller does not only need a flexible function approximator. It also needs a structured decision process in which some variables must satisfy known relationships and bounds.

Efficiency is another issue when this type of layer is used in practice. A QP layer is not as cheap as a standard fully connected layer, because the forward pass needs to solve an optimization problem and the backward pass needs to differentiate through the solution. More recent work on QP Layer focuses on improving the differentiation of convex quadratic optimization problems [12]. For this project, this line of work helps explain why numerical stability, solver tolerance, and QP size are practical design concerns rather than only implementation details.

The difference from the baseline MLP is therefore quite clear. The MLP directly maps the input state to a control force. The OptNet-QP model instead uses the neural network to help construct the QP objective, and the final control output is taken from the QP solution. In this way, the output is affected not only by learned parameters, but also by the constraints written into the optimization problem. This is the main reason why the OptNet-QP layer is more suitable for the constrained-control focus of this thesis.

2.5 Differentiable MPC and Receding-Horizon Learning

The OptNet-QP controller in this thesis is also inspired by differentiable MPC. Amos et al. proposed differentiable MPC for end-to-end planning and control, where the optimization process itself can be included in a learning system [5]. The key idea relevant to this project is that a control decision does not have to be produced by a plain neural-

network output only. It can also be obtained through an optimization module that considers predicted future behaviour.

This project uses that idea in a smaller form. It does not implement a full nonlinear differentiable MPC controller. Instead, the model uses a finite-horizon QP layer. At each time step, the QP predicts a short sequence of future states and control inputs, but only the first control input is used as the final output. This follows the receding-horizon principle of MPC, where the optimization is repeated when new state information becomes available [6].

The length of the prediction horizon is therefore a practical design choice. A longer horizon may give the controller more future information, but it also increases the number of QP variables and constraints. This makes the layer slower and harder to train inside a neural network. Related MPC studies also show that horizon design and model granularity can influence both performance and computational cost [13]. For this reason, the main model uses a 10-step horizon as a reference setting. It is long enough to include short-term future behaviour, but still small enough to keep the QP layer manageable during training.

This choice is later checked through the horizon comparison experiment. The purpose is not to assume that $N = 10$ is always optimal, but to use it as a reasonable starting point and then examine whether other horizons give better rollout behaviour.

2.6 Data Preprocessing for Supervised Control

The generated trajectories need to be checked before they are used for training. Since the dataset comes from simulated cart-pole trajectories, some samples may contain abnormal or extreme motion. If these samples are kept, they can increase the supervised loss and make the controller harder to train. This is why data cleaning is included before model training. Kotsiantis et al. also note that cleaning, transformation, and normalization can affect the performance of supervised learning models [14].

In this project, preprocessing mainly deals with two issues. One issue is trajectory quality. Failed or extreme trajectories are removed because they are not useful as expert demonstrations. The other issue is scale difference. Cart position, velocity, pole angle,

angular velocity, and control force are measured in different units, so their numerical ranges are not naturally comparable. If the raw values are used directly, the training process may become less stable.

The state data and control labels are therefore normalized before training. The mean and standard deviation are calculated only from the training set, and the same values are then applied to the validation and test sets. This keeps the evaluation fair and avoids using information from the validation or test data during preprocessing. For the OptNet-QP model, one extra step is needed. The neural network receives normalized inputs, but the QP constraints still have to be written in physical units, such as metres and newtons. Therefore, the model converts the input back to physical scale inside the QP layer before constructing the constraints. Therefore, the input is converted back to physical coordinates inside the QP layer, so that bounds such as cart position and control force remain physically meaningful.

Chapter 3

Methodology

This chapter explains how the constrained cart-pole controller is built and evaluated. The work starts from expert state control trajectories generated by trajectory optimization. These trajectories are then used as demonstrations for supervised imitation learning, where the model learns to predict the expert control input from the current cartpole state.

Two controllers are developed in this project. The first one is a baseline MLP, which directly maps the current state to the control force. This model is simple and useful as a reference, but it does not contain any explicit physical constraint in its output layer. The second one is the proposed OptNet-QP controller. Instead of directly predicting the force, the proposed model produces the control output through a differentiable quadratic programming layer. In this layer, the force limit, cart position bound, and short horizon

predictive relationship are included as part of the optimization problem. This makes the output more structured than a normal neural network prediction.

The reason for comparing this model with the MLP baseline is to check whether the QP layer actually improves constraint satisfaction during rollout. The following sections describe the main steps of the method, including expert trajectory generation, data preprocessing, OptNet-QP model design, baseline MLP design, and the training and evaluation setup. The dataset statistics and trajectory plots are left to Chapter 4, where they are discussed together with the experimental results.

3.1 Overall Project Pipeline

The overall pipeline is:

1. Data Generation

Generate cart-pole trajectories using trajectory optimization, producing state-control pairs as expert demonstrations.

2. Trajectory Filtering and Preprocessing

Remove failed or extreme trajectories based on a predefined threshold to ensure data quality.

3. Train / Validation / Test Split

Randomly split the dataset into training (70%), validation (10%), and testing (20%) sets.

4. Normalization

Normalize state and control data using the mean and standard deviation computed from the training set only, to improve training stability.

5. OptNet-QP Model Design and Training

The main model is designed by combining a neural feature network with a differentiable QP layer. The QP layer imposes control force constraints, cart position bounds, and a linear short-horizon predictive structure.

6. Baseline Design and Training

A standard multilayer perceptron is trained as a comparison model. Unlike the OptNet-

QP model, it directly predicts the control input from the current state without an explicit constrained optimization layer.

7. Evaluation

Evaluate the models using prediction accuracy, rollout behaviour, constraint violation analysis, success-rate metrics, prediction-horizon comparison, and QP matrix visualization.

3.2 Data Generation

The training data are generated from cart-pole trajectories. The cart-pole model and trajectory optimization problem are built using Pinocchio and aligator. Aligator is used in this project as the trajectory optimization tool for generating expert demonstrations. It provides implementations of constrained trajectory optimization methods, including ProxDDP, a proximal differential dynamic programming method for constrained trajectory optimization [10]. In this project, aligator is not the learning model itself. Instead, it is used offline to generate reference state-control trajectories, and these trajectories are then used as supervised learning data for training the MLP and OptNet-QP controllers.

In the data generation stage, the continuous dynamics are discretized with a semi-implicit Euler integrator. The time step is set as: $dt = 0.01s$. Each trajectory contains 500 time steps. Therefore, one trajectory corresponds to approximately 5 seconds of cart-pole motion.

The state of the cart-pole system is represented as:

$$\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]^T$$

where x is the cart position, $\dot{x}$ is the cart velocity, θ is the pole angle, and $\dot{\theta}$ is the pole angular velocity. The control input is denoted as: u , which represents the horizontal force applied to the cart.

During data generation, the cart position is constrained within:

$$-1.0 \leq x \leq 1.0$$

The control signal is also kept within a physically meaningful range:

$$-10 \leq u \leq 10$$

A total of 1000 candidate trajectories are generated. For each trajectory, the initial state is randomly sampled from a bounded range. The initial cart position is sampled from $[-0.3, 0.3]$, the cart velocity from $[-0.5, 0.5]$, the pole angle from $[-0.5, 0.5]$, and the pole angular velocity from $[-1.0, 1.0]$. This sampling range allows the dataset to cover different initial configurations while avoiding unrealistic starting states.

Small Gaussian noise is added to the generated trajectories. The state noise standard deviation is set to 0.01, and the control noise standard deviation is set to 0.1. The purpose of adding noise is to make the dataset less idealized and closer to a practical learning setting. At the same time, the noise level is kept small so that the main physical meaning of the trajectories is preserved.

One important implementation detail is the state order. The trajectory generation environment outputs the state in the order:

$$[x, \theta, \dot{x}, \dot{\theta}]$$

However, the learning models in this project use the order:

$$[x, \dot{x}, \theta, \dot{\theta}]$$

Therefore, the state variables are rearranged before being saved into the final dataset. This ensures that the dataset, the baseline model, and the OptNet-QP model all use the same state representation.

After data generation, a filtering step was applied to remove failed or extreme trajectories before model training. This step is important because abnormal samples may introduce large errors into the training set and reduce the stability of supervised learning, especially when the loss function is based on mean squared error. Data cleaning is a common preprocessing step in supervised learning [14], and outlier detection methods are also commonly used to identify and, when appropriate, remove anomalous observations from a dataset [15].

For each generated trajectory, the maximum absolute value over all time steps and all state dimensions were calculated. A trajectory was kept only if this value was smaller than a predefined threshold. The filtering criterion can be written as

$$\mathcal{I}_{valid} = \left\{ i \mid \max_{t,j} |x_{t,j}^{(i)}| < \tau \right\},$$

where $x_{t,j}^{(i)}$ denotes the j -th state component of the i -th trajectory at time step t , and τ is the filtering threshold. In this project, the threshold was set to $\tau = 20$.

This threshold was used as a practical data-cleaning criterion to remove trajectories with unrealistically large state values. It should be noted that this filtering threshold is not the same as the physical constraints used later in the QP layer. The physical constraints, such as the cart position bound and the control force bound, are part of the control problem, while $\tau = 20$ is only used to remove abnormal trajectories before supervised training.

After filtering, the final dataset contained 994 valid trajectories. Each trajectory contained $T = 500$ time steps. The state dimension was $n_x = 4$, and the control dimension was $n_u = 1$. Therefore, the final state and control datasets were stored as

$$X \in \mathbb{R}^{994 \times 500 \times 4},$$

$$U \in \mathbb{R}^{994 \times 500 \times 1},$$

where X contains the state trajectories and U contains the corresponding control inputs.

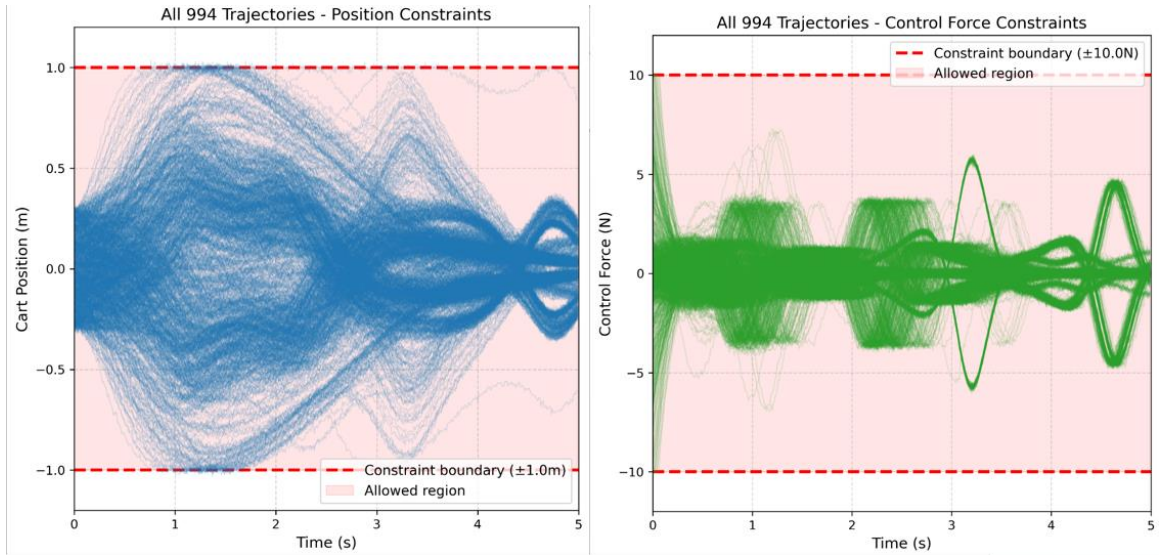


Figure 1. Generated cart-pole trajectories used for supervised learning.

Figure 1 visualizes all 994 generated expert trajectories used in this project. The left subplot shows the cart-position trajectories, where the dashed red lines indicate the prescribed position constraints between -1m and +1m. The right subplot shows the corresponding control-force profiles, with the dashed red lines representing the force limits

between -10N and +10N. As shown in both subplots, the generated demonstrations remain within the admissible regions throughout the rollout horizon.

These trajectories therefore provide constraint-satisfying expert demonstrations for supervised learning, allowing the models to learn the mapping from the current cart-pole state to the corresponding control input while preserving the physical constraint structure of the task.

3.3 Data Preprocessing and Normalization

Before training, the dataset is reshaped to match the model input and output format. The state tensor is converted from:

$$(994 \cdot 500 \cdot 4)$$

to:

$$(994 \cdot 500 \cdot 4 \cdot 1)$$

and the control tensor is converted from:

$$(994 \cdot 500 \cdot 1)$$

to:

$$(994 \cdot 500 \cdot 1 \cdot 1)$$

The additional final dimension is used to keep the tensor format consistent in the PyTorch implementation.

The dataset is then randomly shuffled and divided into training, validation, and testing sets. The split ratio is:

$$70\%:10\%:20\%$$

The training set is used to update the model parameters. The validation set is used for model selection and learning rate adjustment. The testing set is used for final evaluation.

Normalization is applied to both the state and control data. The mean and standard deviation are computed only from the training set:

$$\mu_x = \text{mean}(X_{train}), \sigma_x = \text{std}(X_{train})$$

$$\mu_u = \text{mean}(U_{train}), \sigma_u = \text{std}(U_{train})$$

The normalized data are calculated as:

$$\hat{X} = \frac{X - \mu_x}{\sigma_x}$$

$$\hat{U} = \frac{U - \mu_u}{\sigma_u}$$

The same training-set statistics are applied to the validation and testing sets. This avoids data leakage and keeps the evaluation fair. Data preprocessing and normalization are important in supervised learning because they reduce the influence of scale differences between variables and improve training stability [14].

For the OptNet model, normalization requires additional care. The neural network receives normalized states as input, but the QP constraints must be defined in physical units. For example, the cart position bound $[-1,1]\text{m}$ and the control force bound $[-10,10]\text{N}$ are meaningful only in physical coordinates. Therefore, the model stores the normalization statistics internally. During the forward pass, the normalized input state is converted back to physical units before the QP constraints are constructed. After the QP is solved, the first control input is converted back to normalized form so that it can be compared with the normalized training label.

This design allows the neural network part to benefit from normalized training, while the QP layer still enforces constraints in physical units.

3.4 OptNet Model Design

The proposed model is named OptNetCartPoleMPC. Its overall structure is shown in Figure 2. The model combines a neural feature extractor with a differentiable QP layer. The normalized state input is first passed through the feature network to generate the state-dependent linear term $p(\mathbf{x})$ of the QP objective. In parallel, the same normalized state is converted back to physical units so that equality constraints and inequality constraints can be constructed using physically meaningful values. This design allows the neural network to be trained in normalized coordinates, while the QP layer still enforces cart position and

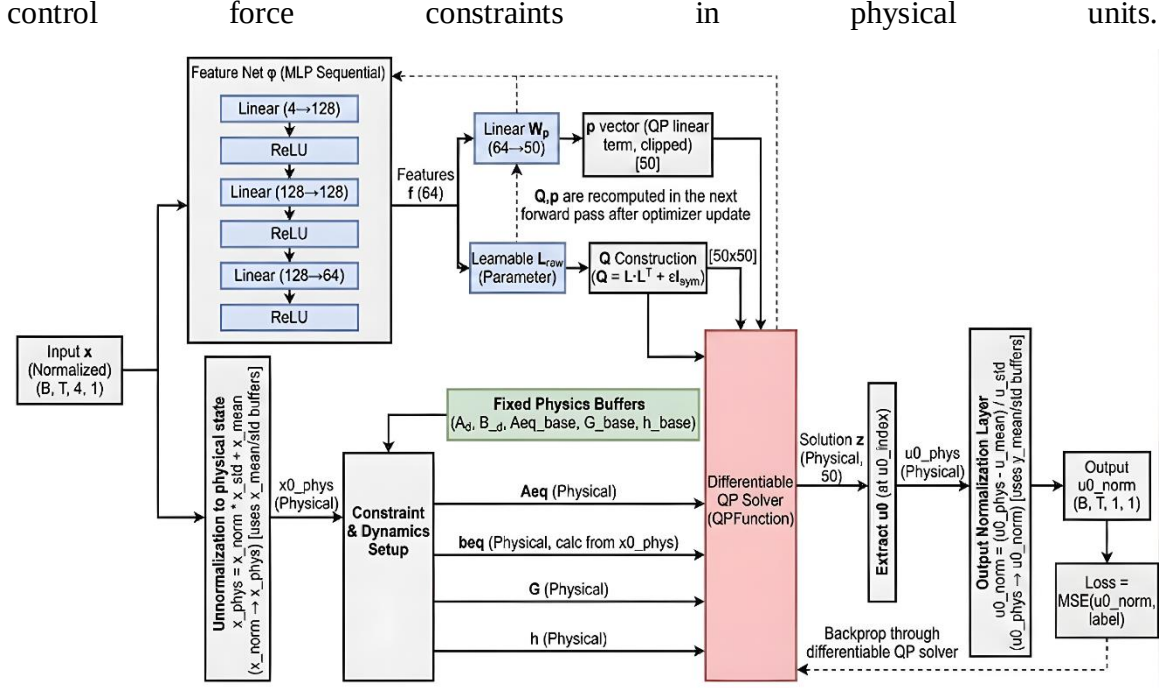


Figure 2. Structure of the proposed OptNetCartPoleMPC model

The model follows the general idea of OptNet, where a QP is solved during the forward pass and gradients are propagated through the QP solution during training [7]. This allows the QP to become part of an end-to-end trainable neural network. The model is also related to the idea of differentiable MPC, where optimization-based control modules are integrated with learning systems [5]. However, this project does not implement a full nonlinear MPC controller. Instead, it uses a smaller MPC-inspired QP layer.

The prediction horizon of the QP is set to:

$$N = 10$$

At each time step, the QP predicts 10 future states and 10 future control inputs. The QP decision variable is:

$$z = [x_1 \ x_2 \ \cdots \ x_N \ u_0 \ u_1 \ \cdots \ u_{N-1}]^T$$

where each predicted state $x_i \in \mathbb{R}^4$, and each control input $u_i \in \mathbb{R}$. Since $N = 10$, the total number of decision variables is:

$$n_z = 10 \times 4 + 10 \times 1 = 50$$

The QP is formulated as:

$$\min_z \frac{1}{2} z^T Q z + p(\mathbf{x})^T z$$

subject to:

$$A_{eq} z = b_{eq}$$

$$G z \leq h$$

where Q is the quadratic cost matrix, $p(\mathbf{x})$ is the state-dependent linear cost vector, A_{eq} and b_{eq} describe the equality constraints, and G and h describe the inequality constraints.

The current state is first processed by a neural feature network: $\phi(\mathbf{x})$

The feature network has the structure:

$$4 \rightarrow 128 \rightarrow 128 \rightarrow 64$$

The output of this feature network is mapped to the linear term of the QP objective:

$$p(\mathbf{x}) = W_p \phi(\mathbf{x})$$

This means that the neural network does not directly output the control force. Instead, it learns how to construct the linear cost term of the QP according to the current state.

To keep the QP convex, the quadratic matrix Q is parameterized as:

$$Q = LL^T + \epsilon I$$

where L is a learnable lower-triangular matrix, I is the identity matrix, and ϵ is a small positive regularization term. In this project, the regularization term is:

$$\epsilon = 10^{-1}$$

This parameterization helps ensure that Q remains positive semi-definite and improves numerical stability during QP solving.

The equality constraints describe a simplified short-horizon linear dynamics model:

$$x_{k+1} = A_d x_k + B_d u_k$$

where:

$$A_d = \begin{bmatrix} 1 & dt & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & dt \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

and:

$$B_d = \begin{bmatrix} 0 \\ dt \\ 0 \\ dt \end{bmatrix}$$

In the OptNet-QP model, the internal QP time step is: $dt = 0.01$

The first equality constraint is:

$$x_1 = A_d x_0 + B_d u_0$$

where x_0 is the current physical state. For later prediction steps, the constraints are:

$$x_i = A_d x_{i-1} + B_d u_{i-1}, i = 2, 3, \dots, N$$

These equality constraints should not be interpreted as a full nonlinear dynamics model of the cart-pole system. The QP layer used in this project is a convex optimization layer with linear equality and inequality constraints. Its role is to impose a structured and constrained output form on the state-control variables. The nonlinear state-dependent mapping is mainly learned by the neural feature network before the QP layer, while the QP layer enforces the specified control force and cart position bounds.

The inequality constraints represent the control force bounds and the cart position bounds. For each predicted control input:

$$-10 \leq u_i \leq 10$$

For each predicted state, the cart position component is constrained by:

$$-1.0 \leq x_i^{pos} \leq 1.0$$

These constraints are written in the standard form:

$$Gz \leq h$$

The use of explicit inequality constraints is the main structural difference between the OptNet model and the baseline MLP. The MLP can only learn the constraint behavior indirectly from data, while the OptNet model places the known physical bounds directly inside the QP layer. This is consistent with the idea of using constraints and domain knowledge to guide deep learning models [2], [3], [4].

Although the QP solves for a sequence of future control inputs,

$$u_0, u_1, \dots, u_{N-1}$$

only the first control input u_0 is used as the model output. This follows the receding-horizon idea used in MPC, where an optimization problem is solved over a finite horizon, but only the first action is applied before the process is repeated at the next time step [5], [6]. A 10-step horizon is used as a practical balance between including short-term future information and keeping the QP layer computationally manageable. Horizon length and model granularity are known to affect the computational cost and performance of MPC-style methods [13].

3.5 Baseline Neural Network

A baseline multilayer perceptron is implemented as the comparison model. The purpose of the baseline is to show how a standard supervised learning controller performs when no explicit physical constraints are included in the model structure.

The baseline model takes the normalized cart-pole state as input and directly predicts the normalized control input. Its network structure is:

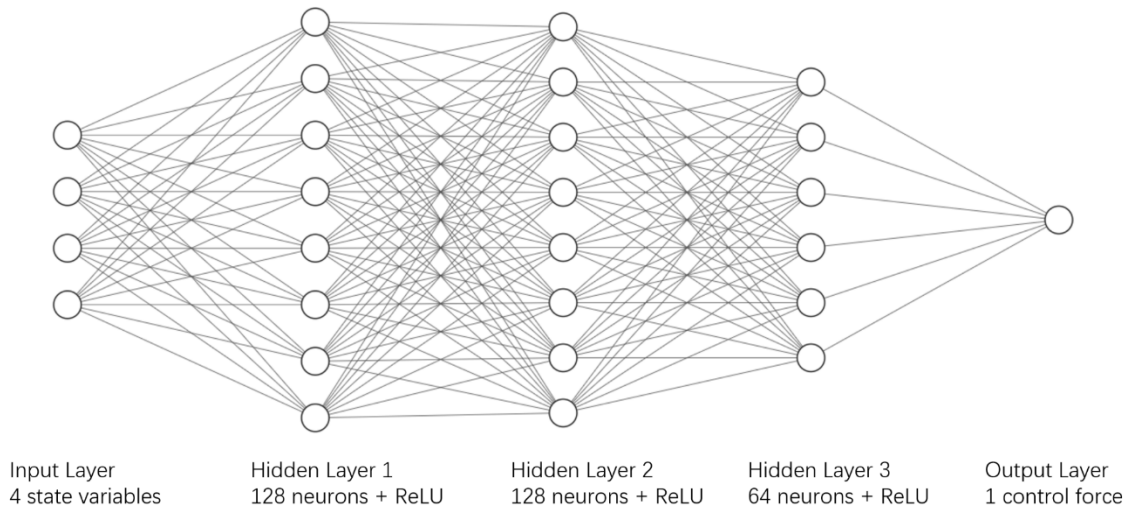


Figure 3. Structure of the baseline MLP

As shown in Figure 3, the baseline MLP directly maps the current cart-pole state to one control output. The input layer contains four state variables, and the output layer predicts the control force. The three hidden layers contain 128, 128, and 64 neurons,

respectively, with ReLU activation functions between them. This structure is simple and efficient, but it does not explicitly include the cart position bound, control force bound, or system dynamics.

ReLU activation functions are used between the hidden layers. The model input has the shape:

$$(B, T, 4, 1)$$

where B is the batch size and T is the trajectory length. Inside the model, the input is reshaped into:

$$(BT, 4)$$

The MLP then outputs:

$$(BT, 1)$$

which is reshaped back into:

$$(B, T, 1, 1)$$

The baseline model is trained using mean squared error between the predicted normalized control input and the normalized expert control input:

$$L = \frac{1}{M} \sum_{i=1}^M (\hat{u}_i - \hat{u}_i^{expert})^2$$

where M is the number of training samples, $\hat{u}_i$ is the predicted control input, and $\hat{u}_i^{expert}$ is the expert control label.

The baseline MLP is simple, fast to evaluate, and useful for measuring the basic supervised learning performance. However, it works mainly as a black-box function approximator. It does not explicitly include the control force bound, the cart position bound, or the short-term system dynamics. Therefore, even if the MLP achieves a low prediction error, it may still violate constraints during rollout. This is the main reason why the OptNet-QP model is introduced as a more structured alternative.

3.6 Training and Evaluation Setup

Both the baseline MLP and the OptNet-QP model are trained using supervised learning. The training target is the normalized expert control input, and the loss function is mean squared error:

$$L = \frac{1}{M} \sum_{i=1}^M (\hat{u}_i - \hat{u}_i^{expert})^2$$

The main training hyperparameters are shown in Table 2.

Table 2. Training hyperparameters

Hyperparameter	Value
Batch size	32
Number of epochs	50
Learning rate	1e-3
Optimizer	Adam
Weight decay	1e-4
Gradient clipping	max norm = 5.0
Learning rate scheduler	ReduceLROnPlateau
Scheduler factor	0.5
Scheduler patience	5
OptNet prediction horizon	10
QP maximum iteration	500
QP tolerance	1e-10

The Adam optimizer is used for parameter updates. Weight decay is used as L2 regularization. To improve the training stability, gradient clipping is applied, particularly for the OptNet model, where the gradients have to go through the QP layer. The learning rate is modified by a ReduceLROnPlateau scheduler that decreases the learning rate when the validation loss stops improving.

During the training we record the training, validation and testing loss after each epoch. We choose the best model according to the validation loss. This means that the final checkpoint is not only the last epoch but the best model on the validation set.

The evaluation includes both prediction accuracy and rollout behavior. For prediction accuracy, the model outputs are converted back to physical units before calculating the final metrics. The metrics include mean squared error, root mean squared error, mean absolute error, R^2 , and Pearson correlation:

$$\begin{aligned}
MSE &= \frac{1}{M} \sum_{i=1}^M (u_i - u_i^{expert})^2 \\
RMSE &= \sqrt{MSE} \\
MAE &= \frac{1}{M} \sum_{i=1}^M |u_i - u_i^{expert}| \\
R^2 &= 1 - \frac{\sum_i (u_i^{expert} - u_i)^2}{\sum_i (u_i^{expert} - \bar{u}^{expert})^2}
\end{aligned}$$

The rollout evaluation is used to test how the trained policy behaves when applied repeatedly over time. This is important because a model with low one-step prediction error may still behave poorly during long-term rollout. In particular, the cart position constraint is evaluated by counting the number of time steps where:

$$x_t > x_{max}$$

or:

$$x_t < x_{min}$$

The constraint violation count is used to compare the OptNet-QP model with the baseline MLP. Since the OptNet model explicitly includes the cart position constraint in the QP, it is expected to produce fewer violations during rollout.

The OptNet-QP model also allows the QP formulation to be checked directly. The matrices Q , p , A_{eq} , b_{eq} , G , and h define the optimization problem solved inside the OptNet layer. By inspecting these terms, it is possible to check whether the simplified dynamics constraints and the inequality constraints have been built correctly. This gives

the model an extra level of interpretability, since the constraint structure can be examined directly instead of being hidden only inside neural network weights.

Chapter 4

Results and Evaluation

4.1 Training Results

This section first looks at whether the OptNet-QP model can be trained stably on the generated expert trajectories. The loss curves are used to check the basic training behaviour before comparing the model with the MLP baseline.

Figure 4 shows the training, validation, and test losses of the OptNet-QP model over 50 epochs. The loss falls quickly at the beginning of training. The training loss starts at about 0.91 and drops clearly within the first few epochs, which means that the model learns the main state to control relationship early in training. The validation and test losses follow a similar trend, so the improvement is not limited to the training set.

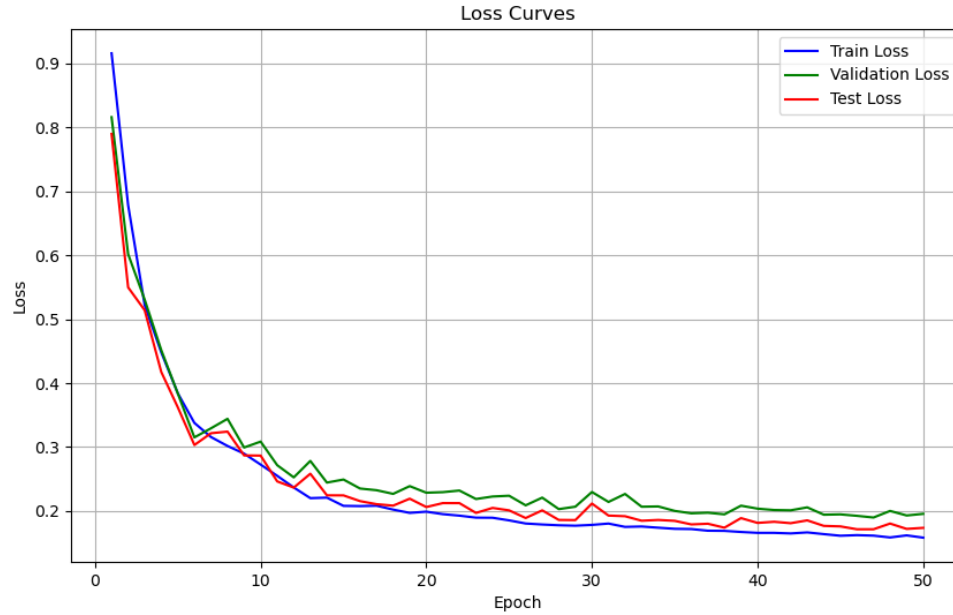


Figure 4. Training, validation, and test loss curves of the OptNet-QP model

After about 15 to 20 epochs, the decrease becomes much slower and the curves begin to level off. The validation loss remains slightly higher than the training loss, which is reasonable because the validation data are not used to update the model parameters. More importantly, the three curves stay close to each other. From this result, the OptNet-QP model does not show obvious overfitting under the current training setting.

Figure 5 further compares the training and test MSE losses of the OptNet-QP model and the MLP baseline. Both models show a clear decreasing trend and eventually reach stable loss values. This means that both models can learn the basic mapping from cart-pole states to expert control inputs.

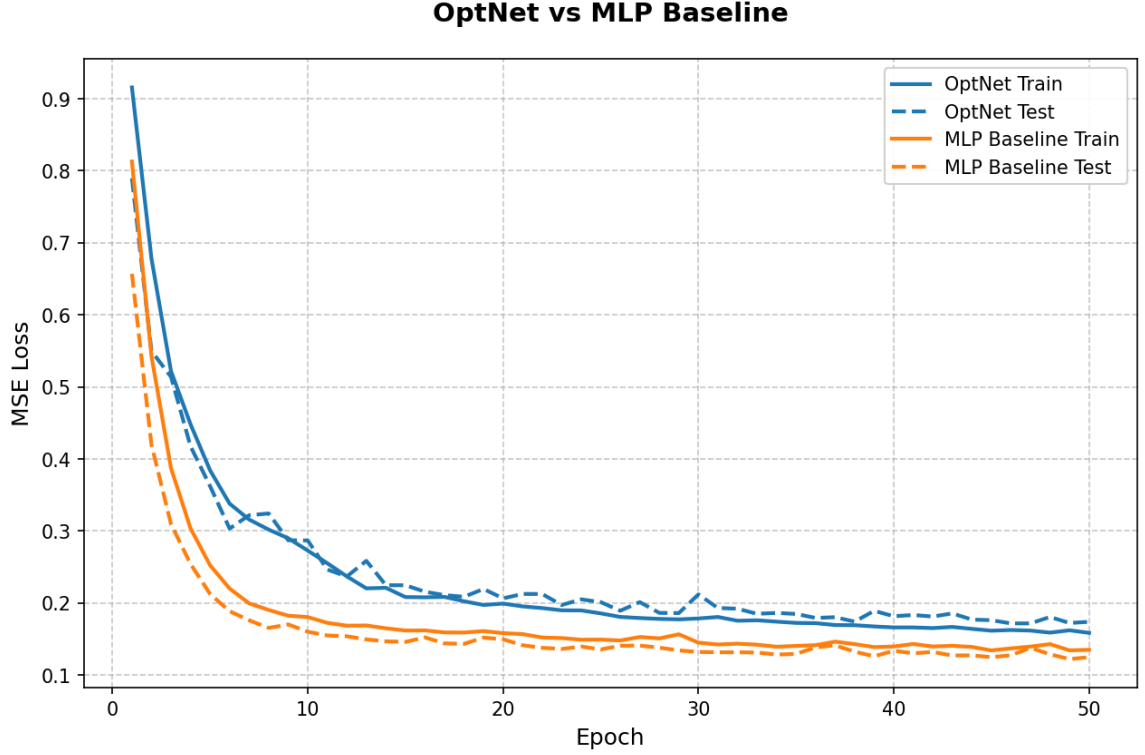


Figure 5. Loss comparison between OptNet-QP and MLP baseline.

One important observation is that the MLP baseline reaches a slightly lower supervised MSE loss than the OptNet-QP model. This is reasonable because the MLP directly optimizes the regression mapping from state to control input. In contrast, the OptNet-QP model produces the control output through an additional constrained optimization layer. Therefore, the OptNet model is not only trying to fit the expert control label but also producing its output through a structured QP formulation.

This comparison also highlights an important limitation of the pure supervised baseline. The MLP is trained only to reduce the one-step prediction error between the predicted control input and the expert label. Therefore, it can achieve a low supervised loss without explicitly considering the physical constraints of the control problem. In other words, the supervised loss mainly measures imitation accuracy at individual time steps, but it does not guarantee constraint-satisfying behaviour during rollout. This is especially important for the cart-pole task, because small prediction errors may accumulate over time and cause the cart position to move outside the allowed range. For this reason, rollout

evaluation is necessary to compare the actual constraint behavior of the MLP baseline and the OptNet-QP model.

To further examine how the explicit cart position constraint affects the supervised training behaviour of the OptNet-QP model, an additional relaxed-bound OptNet-QP model is compared with the constrained OptNet-QP model and the MLP baseline. The relaxed-bound OptNet-QP model keeps the same QP structure as the constrained model, including the neural feature network, QP objective, simplified dynamics constraints, and control force bounds. However, the cart position bound is relaxed to a very large value, $x_{\max} = 100.0$, so that the cart position constraint is effectively inactive. Therefore, this model is not a fully unconstrained controller, but it provides a useful comparison for analysing the influence of the cart position constraint on the supervised validation loss.

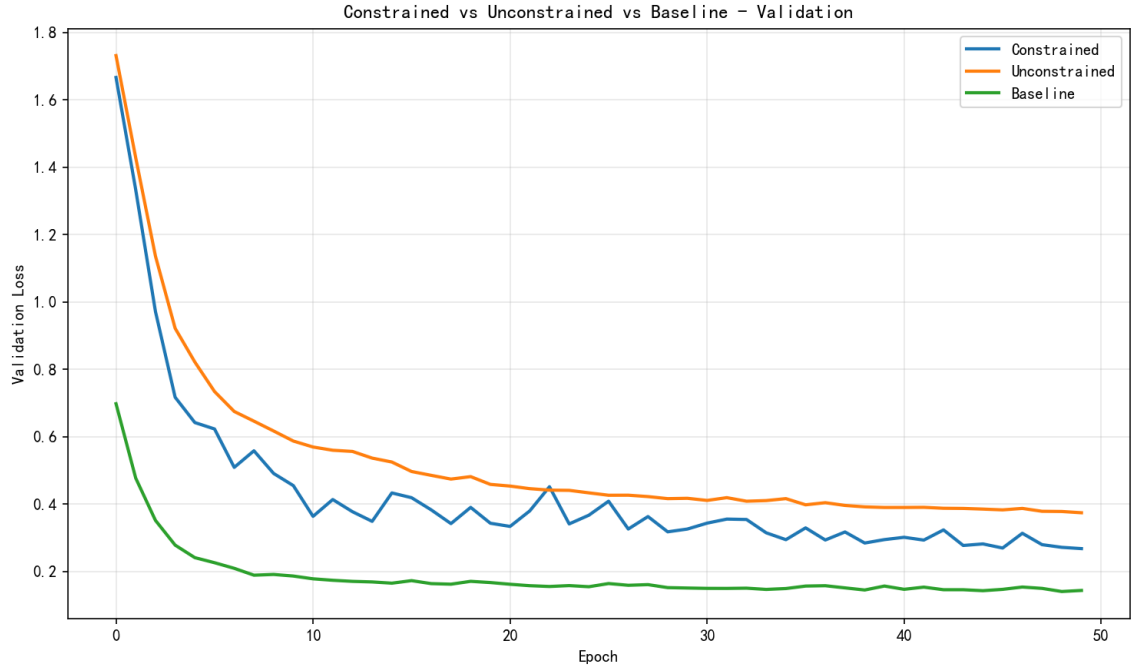


Figure 6. Validation loss comparison among constrained OptNet-QP, relaxed-bound OptNet-QP, and MLP baseline.

As shown in Figure 6, the relaxed-bound OptNet-QP model does not achieve a lower validation loss than the constrained OptNet-QP model. This result may seem counter-intuitive at first, because relaxing the cart position bound gives the QP a larger feasible region. However, the validation loss reported here is not the objective value of the QP

itself. It is the mean squared error between the first predicted control input and the expert control label. Therefore, a larger feasible region does not necessarily make the predicted control input closer to the expert label.

This result suggests that the explicit cart position constraint may provide a useful physical inductive bias during supervised training. Since the generated expert trajectories are produced under physically meaningful cart position limits, keeping this constraint inside the QP layer can help the model learn a control behaviour that is more consistent with the structure of the expert data. At the same time, this comparison further confirms that supervised loss alone should not be used as the only criterion for judging a constrained controller. Rollout performance, constraint violation count, and success-rate evaluation are still needed to assess the actual closed-loop behaviour of the learned models.

4.2 Prediction Accuracy

The prediction results are reported in physical units rather than only in normalized loss values. During training, the control labels are normalized to make the optimization more stable, but the normalized scale is not very helpful when discussing the actual force applied to the cart. Therefore, before calculating the final results, both the predicted control inputs and the expert labels are converted back to their original physical scale.

The evaluation uses MSE, RMSE, MAE, R^2 , and Pearson correlation. These metrics are not used to judge the controller alone, but they give a basic check of how closely the OptNet-QP output follows the expert control signal. MSE and RMSE reflect the size of the prediction error, MAE shows the average absolute difference, R^2 describes how much of the expert signal variation is captured, and Pearson correlation checks whether the predicted force follows a similar trend.

The results are shown in Table 3.

Table 3. Prediction accuracy of the OptNet-QP model in physical units

Metric	Train	Validation	Test
MSE	0.3169	0.4009	0.2932
RMSE	0.5630	0.6331	0.5415
MAE	0.2916	0.3071	0.2847
(R^2)	0.8413	0.8046	0.8534
Corr	0.9180	0.8973	0.9244

The test set results show that the OptNet-QP model reaches an MSE of 0.2932 and an RMSE of 0.5415 in physical units. The MAE is 0.2847, which means that the average absolute difference between the predicted control input and the expert control input is relatively small. The test R^2 value is 0.8534, which indicates that the model can explain a large proportion of the variation in the expert control signal. The correlation coefficient is 0.9244, showing that the predicted control values follow the expert control trend closely.

The validation performance is slightly weaker than the training and test performance. This may be related to the distribution of trajectories in the validation split, since the dataset is divided at the trajectory level and different initial conditions may produce slightly different control behaviours. Even so, the validation R^2 remains above 0.80, and the validation correlation remains close to 0.90. Therefore, the model still shows a reasonable generalization ability.

These results show that the OptNet-QP model can learn the expert control mapping with acceptable accuracy. However, prediction accuracy alone is not enough to evaluate a controller. A controller may predict the expert control input well at individual time steps, but still produce poor behaviour when it is rolled out over time. For this reason, rollout performance and constraint satisfaction are also evaluated in the following sections.

4.3 Rollout Performance

The main purpose of this project is to test whether the OptNet-QP model can better respect physical constraints than a standard MLP baseline. Therefore, a rollout comparison is conducted between the OptNet model and the baseline MLP under the same cart position constraint.

Figure 7 shows the rollout trajectories of the two models under the cart position limit:

$$-1.0 \leq x \leq 1.0$$

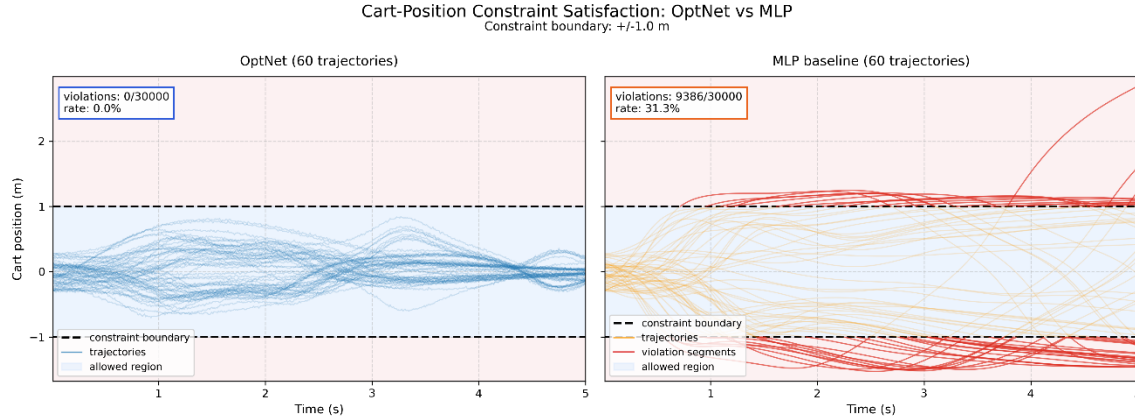


Figure 7. Rollout comparison between OptNet and MLP under the ± 1.0 m cart position constraint

In the figure, the black dashed lines represent the upper and lower cart position constraints. The light curves represent individual rollout trajectories, and the red segments highlight the parts where the MLP trajectory violates the cart-position constraint. The OptNet-QP model produced 0 constraint violations across all evaluated rollout steps, while the MLP baseline produced 9386 violations out of 30000 total rollout steps. In contrast, the MLP baseline quickly moves outside the lower constraint boundary and remains outside the allowed range for a large part of the rollout.

The constraint violation count is summarized in Table 4.

Table 4 . Cart position constraint violations under the ± 1.0 m constraint

Model	Violation count	Total rollout steps	Violation rate
OptNet-QP	0	30000	0%
MLP baseline	9386	30000	31.3%

Table 4 shows that the constraint behaviour of the two controllers is quite different. The OptNet-QP model stays within the tested cart position boundary for all evaluated rollout steps. In contrast, the MLP baseline violates the boundary 9386 times out of 30000 steps, which corresponds to a violation rate of 31.3%. This suggests that the lower supervised loss of the MLP does not translate into better rollout constraint satisfaction.

This result directly supports the main motivation of the project. The baseline MLP can learn a state-to-control mapping from data, but it does not have an explicit mechanism to enforce the cart position bound. Once the predicted action leads the state outside the safe range, the MLP has no built-in constraint layer to correct the trajectory. The OptNet model behaves differently because the cart position bound is included directly in the QP formulation. At each prediction step, the QP layer solves a constrained optimization problem, so the predicted short-horizon states are restricted by the inequality constraints.

This does not mean that the OptNet model is a complete replacement for a full nonlinear MPC controller. The internal dynamics used in the QP layer are still simplified, and the rollout depends on how well the learned objective and the simplified dynamics match the actual system behaviour. However, compared with the MLP baseline, the OptNet-QP model provides a much stronger constraint-satisfaction mechanism. This result is consistent with the basic idea of using optimization layers and domain constraints to improve neural network decision-making [2], [3], [4].

4.4 Success Rate Evaluation and Prediction-Horizon Comparison

In addition to prediction accuracy and cart position constraint violation count, a success-rate metric is used to evaluate the closed-loop rollout performance of the learned OptNet-QP controllers. The violation count in Section 4.3 measures how often the cart position moves outside the specified boundary during one rollout. However, this metric only evaluates constraint satisfaction for the cart position. It does not directly show whether

the controller can complete the swing-up task and keep the pole close to the upright position. Therefore, a success-rate criterion is introduced in this section.

In this evaluation, each trained OptNet-QP controller is rolled out for 50 episodes. Each rollout contains 500 simulation steps, with a time step of 0.01 s. A rollout is considered successful if the cart position satisfies $|x| < 1.0$ m and the pole angle satisfies $|\theta| < 0.2$ rad for 25 consecutive time steps. This criterion is used to avoid counting a short accidental moment of stability as a successful rollout. The success rate is calculated as the number of successful rollouts divided by the total number of evaluated rollouts.

It is important to note that this evaluation is different from checking whether the original expert trajectories are successful. The expert trajectory check directly evaluates the trajectories stored in the generated dataset. In contrast, the success-rate evaluation in this section uses only the initial states from the dataset. The trained OptNet-QP model then predicts the control input at each time step, and the system is rolled out in closed loop. Therefore, this evaluation measures the actual closed-loop control ability of the learned model rather than the quality of the original expert demonstrations.

Table 5 shows the success-rate results for OptNet-QP models with different prediction horizons. In addition to success rate, the table also reports validation loss, rollout RMSE, cart position violation rate, and maximum violation magnitude. These metrics are included because success rate is a binary criterion and may not fully describe the severity of failed rollouts.

Table 5. Performance comparison of OptNet-QP models with different prediction horizons

Horizon N	Validation loss	Rollout RMSE	Violation rate	Max violation	Success rate
1	0.3053	3.5815	10.00%	0.1384	38.00%
2	0.2897	3.6952	24.00%	0.0681	34.00%
5	0.3096	3.5761	2.00%	0.0784	32.00%
8	0.2838	3.6342	0.00%	0.0000	30.00%

Horizon N	Validation loss	Rollout RMSE	Violation rate	Max violation	Success rate
10	0.2802	3.6607	6.00%	0.1167	32.00%
12	0.3092	3.5336	0.00%	0.0000	40.00%
15	0.2964	3.5883	0.00%	0.0000	36.00%
18	0.2872	3.5209	0.00%	0.0000	32.00%

As shown in Table 5, the highest success rate is achieved by the model with $N = 12$. This model reaches a success rate of 40.00%, while also keeping the cart position violation rate at 0.00% and the maximum violation at 0.0000. Therefore, among the tested prediction horizons, $N = 12$ provides the best overall balance between closed-loop success and constraint satisfaction.

The results also show that the validation loss does not directly determine the closed-loop rollout performance. For example, the model with $N = 10$ achieves the lowest validation loss of 0.2802, but its success rate is only 32.00% and its violation rate is 6.00%. In comparison, the model with $N = 12$ has a higher validation loss of 0.3092, but it achieves the highest success rate and no cart position violation. This means that one-step supervised imitation loss is useful for measuring prediction accuracy, but it is not sufficient for evaluating the actual control ability of the learned policy.

The effect of the prediction horizon is also not monotonic. Increasing N does not always improve the success rate. For example, $N = 18$ achieves the lowest rollout RMSE of 3.5209 and also has 0.00% violation rate, but its success rate is only 32.00%. Similarly, $N = 8$, $N = 15$, and $N = 18$ all maintain 0.00% cart position violation, but their success rates are different. This suggests that cart position constraint satisfaction and full swing-up success are related but not identical evaluation targets.

This result is reasonable because the cart position constraint is explicitly included in the QP inequality constraints. As a result, several horizon settings can keep the cart position within the tested boundary. However, the success-rate metric also requires the pole angle to remain close to the upright position for a continuous period. Pole angle stabilization is

more strongly affected by the accuracy of the dynamics model and by the long-term closed-loop behaviour. Therefore, a model can satisfy the cart position constraint while still failing the full swing-up success condition.

One important reason for the limited success rate is the simplified prediction model inside the QP layer. The cart-pole swing-up task is nonlinear, while the QP layer uses a linear predictive structure. This design keeps the optimization problem small enough for supervised training, but it cannot fully represent the nonlinear rollout dynamics. During rollout, even a small mismatch between the internal QP model and the evaluation environment can accumulate over many steps. In this case, the controller may still keep the cart position inside the allowed range, but the pole angle may not stay close enough to the upright position to satisfy the success condition.

The training objective is another reason for this behaviour. The models are trained with single step supervised imitation loss rather than a direct rollout objective. During closed loop evaluation, each predicted control input changes the next state, and the next prediction is then based on that new state. If small errors appear repeatedly, the trajectory can gradually move away from the expert data distribution. This problem is common in imitation learning: a policy may match expert actions well on the dataset but become less reliable once it is rolled out by itself [16].

The success rate results therefore show a mixed outcome. The current OptNet-QP controller is effective at reducing cart position violations, but full swing up stabilization is still difficult. Among the tested horizons, $N = 12$ gives the best result under the current evaluation setting, with a success rate of 40.00% and a cart position violation rate of 0.00%. This is why $N = 12$ is treated as the most suitable horizon in this experiment, even though $N = 10$ has the lowest validation loss. The comparison also shows that the horizon should be selected from rollout behaviour, not from validation loss alone. Future work should therefore focus on a more accurate internal dynamics model, a smaller gap between training and rollout evaluation, and training objectives that directly consider multi step behaviour.

4.5 QP Constraint Verification

To further explain why the OptNet-QP model can better satisfy the cart position constraint during rollout, the inequality constraint matrix G and the constraint vector h are visualized in this section. These two terms define the inequality constraints in the standard QP form:

$$Gz \leq h$$

where z is the QP decision variable containing the predicted future states and control inputs over the 10-step prediction horizon.

In this project, the QP decision variable is arranged as:

$$z = [x_1, x_2, \dots, x_{10}, u_0, u_1, \dots, u_9]^T$$

where each predicted state x_i contains four components. The cart position constraint is applied only to the position component of each predicted state, denoted as $x_i[pos]$. The control force constraint is applied to each predicted control input u_i .

Figure 8 shows the sparse structure of the inequality constraint matrix G .

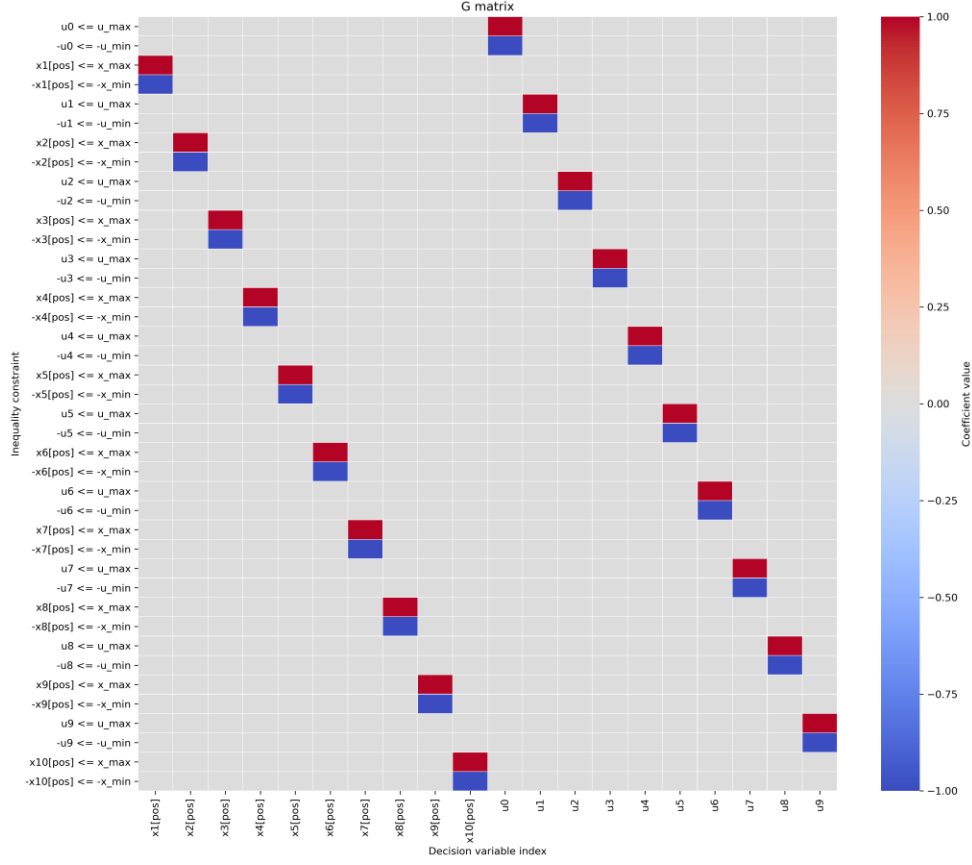


Figure 8. Sparse structure of the inequality constraint matrix G

In Figure 8, the red entries represent coefficient $+1$, while the blue entries represent coefficient -1 . The u_i columns correspond to the predicted control forces, and the $x_i[pos]$ columns correspond to the cart position components of the predicted states. For readability, only the columns related to cart position and control input are shown.

The positive coefficients represent upper-bound constraints, such as:

$$u_i \leq u_{max}$$

and:

$$x_i[pos] \leq x_{max}$$

The negative coefficients represent lower-bound constraints, such as:

$$-u_i \leq -u_{min}$$

and:

$$-x_i[pos] \leq -x_{min}$$

Therefore, the G matrix shows that both the control force bounds and the cart position bounds are explicitly encoded in the QP layer. This is different from the MLP baseline, which does not contain a constraint matrix and can only learn constraint-related behaviour indirectly from the training data.

Figure 9 shows the corresponding inequality constraint vector h .

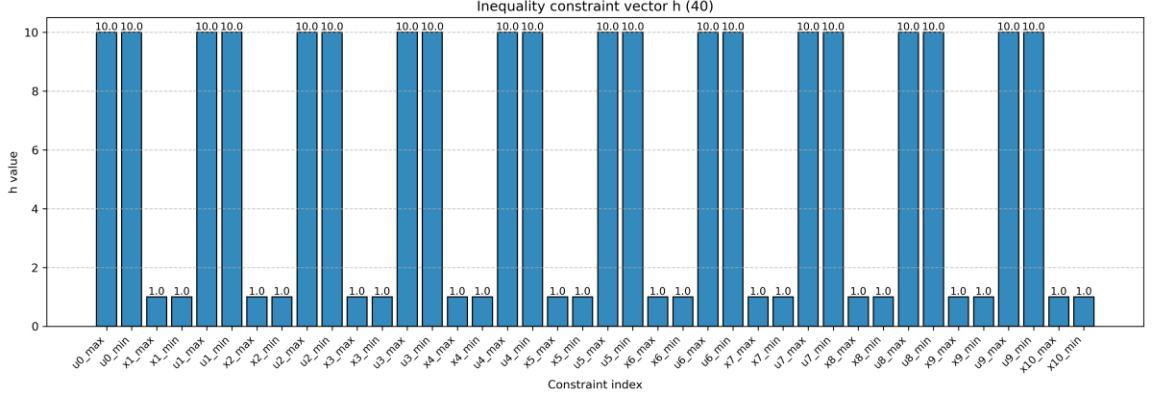


Figure 9. Inequality constraint vector h for the 10-step OptNet-QP model

The entries in h represent the numerical bounds used in the inequality constraints. Since the control force is bounded by:

$$-10 \leq u_i \leq 10$$

the corresponding entries in h are 10 for both the upper and lower control constraints.

This is because the lower-bound constraint $u_i \geq -10$ is rewritten as:

$$-u_i \leq 10$$

Similarly, since the cart position is bounded by:

$$-1 \leq x_i[pos] \leq 1$$

the corresponding entries in h are 1 for both the upper and lower position constraints.

This explains why the h vector contains repeated values of 10 and 1.

Together, Figure 8 and Figure 9 verify that the physical constraints are not only learned from data but are directly included in the QP formulation. The OptNet-QP model can therefore restrict its predicted short-horizon trajectory through the inequality constraints $Gz \leq h$. This helps explain the rollout result in Section 4.3, where the OptNet-QP model stays within the tested cart position boundary, while the MLP baseline violates the same constraint for a large part of the rollout.

It should also be noted that G and h are fixed constraint terms rather than learned parameters. The learnable parts of the OptNet-QP model mainly affect the QP objective, such as the quadratic cost matrix Q and the state-dependent linear term $p(\mathbf{x})$. This distinction is important because the constraint satisfaction behaviour comes from the explicit QP formulation, not only from the neural network learning process.

Chapter 5

Conclusion and Future Work

5.1 Conclusion

This thesis studied a constrained supervised learning method for the cart pole control task. The main purpose was to test whether an OptNet-QP controller can learn from expert trajectories while keeping the physical constraints visible in the model structure. For comparison, a standard MLP controller was also implemented under the same supervised learning setting.

The work began with expert cart pole trajectories generated for imitation learning. After removing abnormal or extreme samples, the final dataset contained 994 valid trajectories, and each trajectory had 500 time steps. The state was represented as

$$\mathbf{x} = [x, \dot{x}, \theta, \dot{\theta}]^T$$

and the target output was the corresponding control force u . The baseline MLP learned this mapping directly from state to force. The OptNetCartPoleMPC model followed a different design: it produced the final control output through a differentiable QP layer rather than a normal neural network output layer.

The main value of the OptNet-QP controller is that the physical restrictions are included in the QP formulation. The model uses a 10 step prediction horizon, where the simplified predictive dynamics, control force bounds, and cart position bounds are all written into the optimization problem. This follows the idea of OptNet, where an optimization problem can be embedded as a differentiable layer inside a neural network

[7]. In this thesis, that idea is used to connect supervised learning with explicit constraint handling.

The prediction results show that the OptNet-QP model can still imitate the expert control signal with reasonable accuracy. In physical units, the test results are $MSE = 0.2932$, $RMSE = 0.5415$, $MAE = 0.2847$, $R^2 = 0.8534$, and $correlation = 0.9244$. These values show that the OptNet-QP model can follow the expert force signal reasonably well on unseen test data. However, the prediction metrics only describe imitation accuracy at individual time steps. They do not fully show whether the controller can behave reliably when it is used repeatedly in rollout.

The rollout result is therefore more closely related to the main aim of this thesis. Under the ± 1.0 m cart position limit, the OptNet-QP model produces no constraint violations in the evaluated rollout steps. The MLP baseline crosses the same boundary 9386 times. This contrast shows the practical role of the QP layer. The MLP can learn the expert action pattern from data, but it has no built in mechanism to enforce the position bound once the rollout starts. This result supports the main argument of this project: a standard MLP may learn a good prediction function, but it does not guarantee constraint satisfaction during rollout. In contrast, the OptNet model can explicitly encode the cart position and control force bounds through the QP constraints $Gz \leq h$.

The success-rate and prediction-horizon evaluation gives a more detailed view of the closed-loop control performance. Under the adopted success criterion, a rollout is considered successful when the cart position satisfies $|x| < 1.0$ m and the pole angle satisfies $|\theta| < 0.2$ rad for 25 consecutive simulation steps. The main OptNet-QP model uses a 10-step prediction horizon as the reference setting, since this keeps the QP size manageable while still including short-horizon future information. The additional horizon comparison shows that $N = 12$ achieves the highest success rate of 40.00% while maintaining 0.00% cart position violation. In contrast, $N = 10$ achieves the lowest validation loss of 0.2802, but its rollout success rate is lower at 32.00% and it still shows a 6.00% violation rate. This confirms that one-step supervised prediction accuracy alone is not sufficient for evaluating closed-loop control performance.

The visualization of the G matrix and h vector further confirms that the constraints are directly written into the QP layer. The G matrix selects the constrained variables, including the predicted cart positions and control inputs, while the h vector stores the corresponding physical bounds. This makes the OptNet model more interpretable than a pure black-box MLP, because the internal constraint mechanism can be inspected directly.

Overall, this project shows that differentiable optimization can be a useful tool for constrained supervised control. The proposed model does not replace full nonlinear MPC, but it provides a practical middle ground between standard neural network imitation learning and optimization-based control. It keeps the learning ability of neural networks while adding a clear constraint mechanism through the QP layer. The results also show that closed-loop swing-up success is still challenging when the model is trained only with one-step supervised imitation loss and simplified QP output structure. Therefore, the main value of the current OptNet-QP model is its improved constraint awareness and interpretability, while further work is still needed to improve final task success and long-term stabilization.

5.2 Limitations

Although the results are encouraging, this project still has two main limitations.

The first limitation is related to the structure and computational cost of the QP output layer. The cart-pole system has nonlinear dynamics, but the QP layer used in this project is a convex optimization layer with linear equality and inequality constraints. Therefore, the QP layer should not be understood as a standalone nonlinear dynamics model. Instead, the nonlinear state-dependent behaviour is mainly learned by the neural network before the QP layer, while the QP layer is used to impose a constrained structure on the predicted states and control inputs. This design is useful for improving constraint satisfaction, but it also means that the final controller depends on how well the neural network and the QP formulation work together. In addition, solving a QP during the forward pass is more computationally expensive than using a standard feedforward MLP output layer, so the prediction horizon and the size of the QP need to remain manageable.

The second limitation is that the method is evaluated only in simulation. The expert trajectories, training data, and rollout tests are all generated in a simulated cart-pole environment. Therefore, the results show the behaviour of the proposed OptNet-QP controller under the current simulation setting, but they do not demonstrate real-world deployment. No sim-to-real transfer or physical cart-pole experiment has been attempted in this project. If the controller were applied to a real system, modelling errors, actuator delays, sensor noise, and other practical uncertainties would need to be considered.

5.3 Future Work

The most immediate direction is to make the training objective closer to the final rollout evaluation. In the current work, the OptNet-QP controller is mainly trained with single step supervised imitation loss. The predicted control input is compared with the expert label at each time step, which is useful for learning the basic mapping from state to control. However, this training target does not fully describe what happens when the controller repeatedly applies its own predicted actions during rollout.

The horizon comparison in Chapter 4 gives a useful warning for the next stage of this work. The model with the lowest validation loss is not the one with the highest rollout success rate. This means that fitting expert actions at isolated time steps is not enough for choosing a good controller. A better training target should look at what happens after the predicted action is actually applied. For example, the loss could include multi step state errors, pole angle stabilization, or penalties when the simulated rollout moves outside the allowed region. The current OptNet-QP structure could still be kept, but the training objective would be closer to the way the controller is finally evaluated.

The prediction model inside the QP layer is another part that needs improvement. In the current implementation, the QP uses a simplified linear dynamics model. This keeps the optimization problem small and makes training more practical, but it also limits how much nonlinear cart-pole behaviour the QP can represent. A more accurate predictive model, or a learned predictive structure, may reduce the mismatch between the QP

prediction and the rollout environment. The key point is to improve the prediction part without losing the explicit constraint handling of the QP layer.

The horizon design should also be treated more carefully. Making the horizon longer is not always the best answer, because it also increases the number of variables and constraints in the QP. The results in this thesis already show that the effect of horizon length is not monotonic. Related MPC research also suggests that model granularity and prediction horizons with unequal spacing can affect the balance between control performance and computational cost [13]. If the method is later used on faster or larger systems, ADMM based MPC may be worth considering because computational efficiency will become more important for real time constrained control [17].

Another possible extension is to move from pure supervised imitation learning toward reinforcement learning. In this thesis, the controller depends on generated expert trajectories. If those demonstrations do not cover enough situations, the learned policy will also be limited. Combining reinforcement learning with the OptNet-QP structure may allow the controller to improve through interaction with the environment, instead of only copying fixed expert actions. Constraint aware reinforcement learning, including methods that treat constraints as termination conditions, provides a relevant direction for this idea [18].

The method should also be tested beyond the cart-pole benchmark. The current task is useful for showing the basic difference between an MLP output and a constrained QP output, but more complex systems would test the method more seriously. Robotic manipulators, double cart-pole systems, and legged locomotion models involve richer constraints, such as joint limits, actuator limits, contact forces, and safety boundaries. These settings may show more clearly whether differentiable optimization layers can provide practical benefits for learning based control.

The next stage of this work should not simply make the model larger. A more meaningful improvement would be to connect the training loss more closely with rollout behaviour, improve the prediction model inside the QP layer, and evaluate the method on systems where physical constraints play a larger role..

References

- [1] S. Levine and V. Koltun, “Guided policy search,” in *Proc. 30th International Conference on Machine Learning (ICML)*, Atlanta, GA, USA, 2013, pp. 1–9.
- [2] A. Borghesi, F. Baldo, and M. Milano, “Improving deep learning models via constraint-based domain knowledge: A brief survey,” *arXiv preprint arXiv:2005.10691*, 2020.
- [3] J. Kotary, F. Fioretto, P. Van Hentenryck, and B. Wilder, “End-to-end constrained optimization learning: A survey,” in *Proc. Thirtieth International Joint Conference on Artificial Intelligence (IJCAI)*, 2021, pp. 4475–4482, doi: 10.24963/ijcai.2021/610.
- [4] P. Márquez-Neila, M. Salzmann, and P. Fua, “Imposing hard constraints on deep networks: Promises and limitations,” in *Proc. CVPR Workshop on Negative Results in Computer Vision*, 2017, pp. 1–9.
- [5] B. Amos, I. D. Jimenez Rodriguez, J. Sacks, B. Boots, and J. Z. Kolter, “Differentiable MPC for end-to-end planning and control,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [6] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl, *Model Predictive Control: Theory, Computation, and Design*, 2nd ed. Madison, WI, USA: Nob Hill Publishing, 2017.
- [7] B. Amos and J. Z. Kolter, “OptNet: Differentiable optimization as a layer in neural networks,” in *Proc. 34th International Conference on Machine Learning (ICML)*, Sydney, Australia, 2017, pp. 136–145.
- [8] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter, “Differentiable convex optimization layers,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 9558–9570.
- [9] D. H. Jacobson and D. Q. Mayne, *Differential Dynamic Programming*. New York, NY, USA: American Elsevier Publishing Company, 1970.

- [10] W. Jallet, A. Bambade, E. Arlaud, S. El-Kazdadi, N. Mansard, and J. Carpentier, “PROXDDP: Proximal constrained trajectory optimization,” *IEEE Transactions on Robotics*, vol. 41, pp. 2605–2624, 2025, doi: 10.1109/TRO.2025.3554437.
- [11] J. Xu, Z. Zhang, T. Friedman, Y. Liang, and G. Van den Broeck, “A semantic loss function for deep learning with symbolic knowledge,” in *Proc. 35th International Conference on Machine Learning (ICML)*, Stockholm, Sweden, 2018, pp. 5502–5511.
- [12] A. Bambade, F. Schramm, A. B. Taylor, and J. Carpentier, “QPLayer: efficient differentiation of convex quadratic optimization,” HAL preprint hal-04133055, 2023.
- [13] T. Brüdigam, D. Prader, D. Wollherr, and M. Leibold, “Model predictive control with models of different granularity and a non-uniformly spaced prediction horizon,” in *Proc. American Control Conference (ACC)*, 2021, pp. 3876–3881, doi: 10.23919/ACC50511.2021.9482617.
- [14] S. B. Kotsiantis, D. Kanellopoulos, and P. E. Pintelas, “Data preprocessing for supervised learning,” *International Journal of Computer Science*, vol. 1, no. 1, pp. 111–117, 2006.
- [15] V. J. Hodge and J. Austin, “A survey of outlier detection methodologies,” *Artificial Intelligence Review*, vol. 22, no. 2, pp. 85–126, 2004.
- [16] S. Ross, G. J. Gordon, and J. A. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proc. 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, Fort Lauderdale, FL, USA, 2011, pp. 627–635.
- [17] A. Aydinoglu and M. Posa, “Real-time multi-contact model predictive control via ADMM,” in *Proc. IEEE International Conference on Robotics and Automation (ICRA)*, 2022, pp. 3414–3421.
- [18] E. Chane-Sane, P.-A. Leziart, T. Flayols, O. Stasse, P. Souères, and N. Mansard, “CaT: Constraints as terminations for legged locomotion reinforcement learning,” in *Proc. IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 13303–13310.

Appendix: Pseudocode of the Training and Evaluation Pipeline

Algorithm 1 OptNet-QP Training Procedure

Require: Training dataset $\mathcal{D} = \{(x_i, u_i)\}_{i=1}^N$, prediction horizon H , linear dynamics matrices, input bounds $(u_{\min}, u_{\max})$, position bounds $(x_{\min}, x_{\max})$

Ensure: Trained OptNet-QP controller

- 1: Initialize neural network parameters θ
- 2: Initialize learnable QP parameters
- 3: Initialize Adam optimizer
- 4: **for** each training epoch **do**
- 5: **for** each mini-batch (x, u) **do**
- 6: Compute feature representation $f_\theta(x)$
- 7: Construct the QP objective parameters:

$$Q_\theta = L_\theta L_\theta^\top + \epsilon I, \quad p_\theta(x) = W_p f_\theta(x)$$

- 8: Construct QP constraints:

$$Gz \leq h, \quad A_{\text{eq}}z = b_{\text{eq}}(x)$$

where $Gz \leq h$ encodes input and cart-position constraints, and $A_{\text{eq}}z = b_{\text{eq}}(x)$ encodes linearized dynamics.

- 9: Solve the differentiable QP:

$$z^\star = \arg \min_z \frac{1}{2} z^\top Q_\theta z + p_\theta(x)^\top z$$

subject to:

$$Gz \leq h, \quad A_{\text{eq}}z = b_{\text{eq}}(x)$$

- 10: Extract the first predicted control action $\hat{u}$ from $z^\star$
- 11: Compute imitation loss:

$$\mathcal{L} = \|\hat{u} - u\|_2^2$$

- 12: Backpropagate through the differentiable QP layer
 - 13: Update network and QP parameters using Adam
 - 14: **end for**
 - 15: **end for**
 - 16: **return** trained OptNet-QP controller
-

Algorithm 2 Closed-Loop Rollout Evaluation

Require: Trained controller $\pi(\cdot)$, cart-pole simulator, rollout horizon T , number of evaluation rollouts N_{eval} , constraint bounds $(x_{\text{max}}, u_{\text{max}})$

Ensure: Closed-loop trajectories and evaluation metrics

- 1: **for** $k = 1$ to N_{eval} **do**
- 2: Sample initial state $x_0^{(k)}$
- 3: **for** $t = 0$ to $T - 1$ **do**
- 4: Compute control input:

$$u_t = \pi(x_t)$$

- 5: Apply u_t to the cart-pole simulator
- 6: Obtain next state x_{t+1}
- 7: Record state, control, and constraint information
- 8: Check constraint violations:

$$|x_t| > x_{\text{max}} \quad \text{OR} \quad |u_t| > u_{\text{max}}$$

- 9: **end for**
 - 10: **end for**
 - 11: Compute rollout metrics:
 - 12: success rate
 - 13: constraint violation count and violation rate
 - 14: average convergence time
 - 15: closed-loop tracking or final-state error
 - 16: **return** evaluation results
-